IOTA Identity Workshop

In this workshop, we will build a production-ready **University Degree Verification system** using **IOTA Identity**, **Rust**, and **React.** By the end of this workshop, you'll have a complete, end-to-end flow demonstrating how **universities**, **students**, and **employers** interact in a real-world university degree validation scenario — all powered by IOTA Identity. We will build the application piece-by-piece, focusing on the communication flow for each of the four core identity steps (Decentralized Identifier creation, Verifiable Credential issuance, Verifiable Presentation creation, and Verification).

This is a snapshot of how the frontend looks.

This website uses cookies

We use cookies to improve your experience. You can manage your preferences below. Read our Cookie Policy

ACCEPT ALL REJECT ALL

University Degree Verification dApp

Demonstrating the end-to-end lifecycle of a Verifiable Credential using React and a Native Rust API (Axum).

Connection Settings

IOTA Package ID: `0x3249a9b8bba4c0a3d5b4b853da038ddb64ee8491632706c5f48e83397bf3ae07`

API Base URL: `http://localhost:3001/api`

Reset Flow

1. Student Creates Identity (Decentralized Identifier)

Student DID (did:iota:...)

```
Awaiting execution...
```

Create Student Identity

2. University Issues Degree to Student (Verifiable Credential)

Verifiable Credential (VC JWT)

```
Awaiting execution...
```

Issue Degree

3. Student Shows Degree To Employer (Verifiable Presentation)

Verifiable Presentation (VP JWT)

```
Awaiting execution...
```

4. Employer Confirm

Validation Output

```
Awaiting execution...
```

Validate Degree

This website uses cookies

We use cookies to improve your experience. You can manage your preferences below. Read our Cookie Policy

The Problem

Today, verifying university degrees is a slow, manual, and often costly hurdle. Employers must contact universities or rely on centralized third-party services, which is costly and prone to fraud. Fake certificates can easily circulate.

The Opportunity

Universities can issue tamper-proof University Degrees that students control using the IOTA Identity. Students present these credentials to employers, who can instantly verify them on the IOTA network without intermediaries, cutting costs and eliminating fraud.

What You'll Build in this Workshop

We're building a complete, system that connects a modern React frontend to a high-performance Rust backend. You'll learn how the frontend triggers the entire IOTA Identity lifecycle:

- **Student (Holder)**: Creates a **Decentralized Identifier (DID)** and stores it securely.
- **University (Issuer)**: Issues a **Verifiable Credential (VC)** representing the degree.
- **Student**: Packages the credential into a **Verifiable Presentation (VP)** and signs it.
- **Employer (Verifier)**: Verifies the authenticity of the presentation and credentials against the IOTA network.

HERE ARE SOME ALTERNATIVE USE CASES

- **Use Case 2:** IOTA Identity unifies your fragmented digital presence under a single, self-controlled Decentralized Identifier (DID), giving you secure owne
- **Use Case 3:** IOT credentials acros product's authen
- **Use Case 4:** IOT tokens to verifie and ensuring aut

This website uses cookies

We use cookies to improve your experience. You can manage your preferences below. Read our Cookie Policy

Understanding Key Terms

Before diving into the workshop, let's clarify three key IOTA Identity concepts used in our University Degree Verification system. These terms, Decentralized Identifier (DID), Verifiable Credential (VC), and Verifiable Presentation (VP)—are central to how we create, issue, share, and verify a digital university degree securely on the IOTA network.

- **Decentralized Identifier (DID)**: A DID is a unique digital ID, like a secure online passport, that identifies someone or something without relying on a central authority. In this workshop:
  - The **student (Holder)** creates a DID to represent their identity.
  - The **university (Issuer)** has its own DID to prove it's a trusted institution.
  - The **employer (Verifier)** uses DIDs to check who issued and holds the degree.
  - Example: A student's DID might look like `did:iota:abc123...`, proving they are the legitimate owner of their degree.
- **Verifiable Credential (VC)**: A VC is a tamper-proof digital document that proves something about its holder, signed by a trusted issuer. In this workshop:
  - The VC is the **university degree** issued by the university to the student.
  - It contains details like the student's name, degree type (e.g., Bachelor of Science), and issuance date, all cryptographically signed to prevent fraud.
  - Example: The university issues a VC saying, "Jane Doe earned a B.Sc. in Computer Science on May 2025."
- **Verifiable Presentation (VP)**: A VP is a way for the holder to share their VC (or parts of it) with someone else, proving its authenticity. In this workshop:
  - The student creates a VP to **present their degree** to an employer.
  - The VP is signed by the student's DID, proving they are the rightful holder of the degree, and includes the VC from the university.
  - Example: Jane D
    valid without reve

These concepts work tog
the student controls their
employer verifies it instan
build this system by creat

Next Steps: Proceed to th
workshop.

This website uses cookies

We use cookies to improve your experience. You can manage your preferences below. Read our Cookie Policy

Prerequisites

Before starting this workshop, make sure you have:

- **Rust** (latest stable version) installed → Install Rust
- **Cargo** (comes with Rust) for building and running projects
- A **local IOTA network** running with the IOTA Identity contract deployed
  - See Local Network Setup
  - Ensure you can request test tokens from the faucet
- A code editor such as **VS Code**
- (Optional but recommended) Basic familiarity with:
  - **IOTA Decentralized Identifiers (DIDs)**
  - **IOTA Verifiable Credentials (VCs)** and **IOTA Verifiable Presentations (VPs)**

Installing Rust and Cargo

Rust and Cargo are required for the Rust backend of the IOTA Identity workshop. Select your operating system below to view the installation instructions:

macOS Windows

Step 1: Install `rustup` (The Rust Installer)

Open your Terminal application and run the following command to download and start the `rustup` installer:

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

You will see a prompt with installation options:

1. Type `1` (for "Proceed with installation (default)").
2. Press Enter.

Step 2: Configure

After installation, update y
available. For Zsh (the de

```
source $HOME/.cargo/e
```

This website uses cookies

We use cookies to improve your experience. You can manage your preferences below. Read our Cookie Policy

NOTE

> If using Bash, the command might be `source $HOME/.bashrc`. The `rustup` installer will display the correct command upon completion. Restart your terminal if needed.

Step 3: Verify the Installation

Confirm that Rust and Cargo are installed correctly:

1. **Check Rust Version**:

   ```
   rustc --version
   ```

   Expected Output: `rustc 1.XX.0 (long_hash_string)`

2. **Check Cargo Version** (Cargo is the build system and package manager):

   ```
   cargo --version
   ```

   Expected Output: `cargo 1.XX.0 (long_hash_string)`

Step 4: Update (Maintenance)

To ensure the latest stable libraries for IOTA Identity, update your Rust toolchain:

```
rustup update stable
```

Step 5: Local Network Setup

> **TIP**
>
> Use this page to set up a Local IOTA Network

Step 6: Install Visual Studio Code

Visual Studio Code (VS C
and run the Rust and Rea
instructions below to insta

This website uses cookies

We use cookies to improve your experience. You can manage your preferences below. Read our Cookie Policy

macOS Windows

Step 6: Install Visual Studio Code on macOS

1. **Download VS Code**:
   - Visit the Visual Studio Code download page.
   - Click the "macOS" button to download the installer (usually a `.zip` file).
2. **Locate the Downloaded File**:
   - Open your browser's download list (e.g., in Safari, click the downloads icon in the toolbar).
   - Find the downloaded file, typically named `VSCode-darwin-universal.zip`.
3. **Extract the Archive**:
   - Double-click the `.zip` file to extract it, which creates `Visual Studio Code.app`.
   - If the file doesn't extract automatically, check your Downloads folder and unzip it manually.
4. **Move to Applications**:
   - Drag `Visual Studio Code.app` to the `Applications` folder in Finder to make it available in Launchpad.
5. **Open VS Code**:
   - Navigate to the `Applications` folder and double-click `Visual Studio Code.app` to launch it.
   - If prompted with a security warning ("App is from an unidentified developer"), right-click the app, select "Open," and confirm.
6. **Add to Dock (Optional)**:
   - While VS Code is running, right-click its icon in the Dock.
   - Choose **Options > Keep in Dock** to add it for easy access.
7. **Verify Installation**:
   - Open VS Code and ensure the welcome screen appears.
   - Open a terminal i
     `code --version`

Once these steps are com

Next Steps: With VS Cod
project and Architecture f

This website uses cookies

We use cookies to improve your experience. You can manage your preferences below. Read our Cookie Policy

Part 1: Project Setup and Architecture

We will create two separate projects that communicate via HTTP: the React Frontend (the user interface) and the Rust Axum Backend (the identity logic engine).

Step 1.1: Project Initialization

Start by creating the root folder and initializing the two sub-projects.

```
# Create the root folder
mkdir iota-identity-workshop
cd iota-identity-workshop

# 1. Initialize the Rust Backend
mkdir iota-identity-backend
cd iota-identity-backend
cargo init

# 2. Set up utility folder for Rust logic
mkdir -p crates/identity_logic/src

# 3. Initialize the React Frontend
cd ..
mkdir iota-identity-frontend
cd iota-identity-frontend
npm create vite@latest . -- --template react
npm install
npm install lucide-react # For icons
```

Explanation:

- **What it does**: Creates a project structure with a Rust backend (`iota-identity-backend`) and a React frontend (`iota-identity-frontend`). The `cargo init` command sets up a Rust project with a `Cargo.toml` file, while `npm create vite` initializes a React app using Vite for fast development. The `lucide-react` library adds icons for the UI.
- **Why it matters**: This sets up a full-stack environment where the frontend handles user interact
  utility folder (`crates/`
- **Key concepts**:
  - **Rust Backend**: U
    operations.
  - **React Frontend**:
    actions.
  - **Vite**: A modern b

This website uses cookies

We use cookies to improve your experience. You can manage your preferences below. Read our Cookie Policy

Code Scaffold

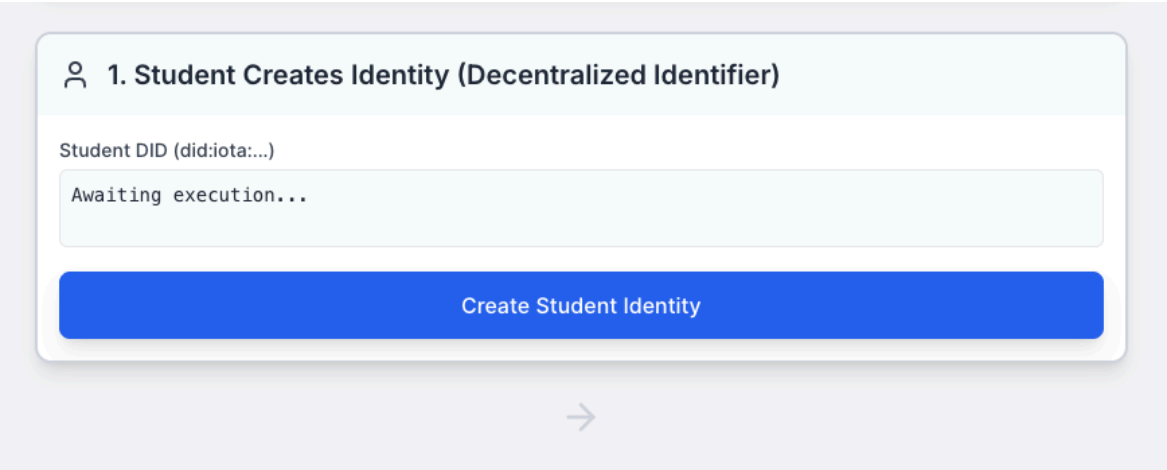
Before starting the IOTA Identity workflow, set up the foundational code for the Rust backend (`main.rs`) and React frontend (`App.jsx`). This scaffold includes imports, data structures, and server/client setup needed to run the project.

> **INFO**
>
> Visit GitHub for the Scaffold Code, copy the code for main.rs and App.jsx.

Next Steps: After setting up the scaffold, return to this workshop to implement the IOTA Identity functionality in Steps 1–4, where you'll add code to create, issue, present, and verify a university degree credential.

Part 2: Building the Front-to-Back Connection

This section demonstrates how every button on the frontend connects to a specific function on the Rust backend, completing the identity flow.



The Communication Layer (`handleApiCall`)

In your **React Frontend** (`src/App.jsx`), this function is the central bridge. It standardizes how all API r
error handling are used fo

This website uses cookies

We use cookies to improve your experience. You can manage your preferences below. Read our Cookie Policy

```
// src/App.jsx (Excer

const handleApiCall =
        if (!packageI
            setError(
the settings.');
            return null;
```

```
        }
        if (!apiUrl) {
            setError('Please set the API Base URL in the settings.');
            return null;
        }

        setLoadingStep(step);
        setError('');

        // This is a minimal exponential backoff retry logic
        const maxRetries = 3;
        const initialDelay = 1000;

        for (let attempt = 0; attempt < maxRetries; attempt++) {
            try {
                // NOTE: The body keys (packageId, vcJwt, vpJwt) MUST
be camelCase to match the #[serde(rename_all = "camelCase")] in Rust.
                const response = await fetch(`${apiUrl}${endpoint}`, {
                    method: 'POST',
                    headers: { 'Content-Type': 'application/json' },
                    body: JSON.stringify({ packageId, ...body }),
                });

                if (!response.ok) {
                    const errorText = await response.text();
                    throw new Error(`API returned status
${response.status}: ${errorText}`);
                }

                const data = await response.json();
                setLoadingStep(0);
                return data;

            } catch (err) {
                if (attempt === maxRetries - 1) {
                    setError(`Error in Step ${step}: ${err.message}`);
                    setLoadingStep(0);
                    return null;
                }
                // Ex
                const
                awaitdelay));
                conso
1}/${maxRetries})...`
            }
        }
    }, [packageId, ap
```

This website uses cookies

We use cookies to improve your experience. You can manage your preferences below. Read our Cookie Policy

Explanation:

- What it does: This function sends HTTP POST requests to the Rust backend's API endpoints (e.g., `/holder/create-did`). It includes a `packageId` (required for IOTA Identity) and handles errors by checking the response status. The result (e.g., a Decentralized Identifier or JWT) is returned to update the frontend state.
- Why it matters: Centralizes API communication, ensuring consistent error handling and payload formatting across all steps. The `useCallback` hook optimizes performance by memoizing the function.
- Key concepts:
  - HTTP POST: Sends data (e.g., `packageId, vcJwt`) to the backend.
  - Error Handling: Catches HTTP errors (e.g., 404, 500) and displays meaningful messages.
  - State Updates: Stores results (e.g., Decentralized Identifier, JWT) in React state for display.

Student Creates Identity

Frontend Action	Rust Endpoint	Payload Sent (Body)	Result Stored (State)
Button Click (`handleCreateDid`)	`/holder/create-did`	`{ "": "..." }`	`did` (string)

Frontend (`src/App.jsx`): We define the handler to call the API endpoint and update the state.

```
// src/App.jsx (Holder DID Handler)

const handleCreateDid = async () => {
    // Calls the Rust endpoint. Body is empty as the DID logic is
self-contained.
    const data = awai
    if (data && data.
        setDid(data.d
    }
};

// This handler is at
```

This website uses cookies

We use cookies to improve your experience. You can manage your preferences below. Read our Cookie Policy

Explanation:

- **What it does**: The `handleCreateDid` function triggers a POST request to the backend's `/holder/create-did endpoint`. It sends an empty body since the backend handles all Decentralized identifier creation logic. The returned Decentralized Identifier string is stored in the React state (`setDid`) for display in the UI.
- **Why it matters**: This initiates the IOTA Identity flow by creating a unique identifier for the student (`Holder`), which is used in later steps (e.g., VC issuance). It connects the frontend button to the backend logic.
- **Key concepts**:
  - **DID**: A decentralized identifier (e.g., `did:iota:...`) that uniquely identifies the student.
  - **React State**: Stores the Decentralized Identifier for display and use in subsequent steps.
  - **API Call**: Uses `handleApiCall` to communicate with the backend.

Backend (`src/main.rs`): The Rust function uses the received `package_id` to initialize the client and runs the core creation logic.

```
// iota-identity-backend/src/main.rs (holder_create_did)
async fn holder_create_did(Json(_body): Json<PackageId>) ->
Result<Json<DidResponse>, StatusCode> {
    let holder_doc_file = "./holder_doc.json";
    let holder_fragment_file = "./holder_fragment.txt";
    let stronghold_path = "./holder.stronghold";

    // This calls create_or_load_did. If the file exists, it loads the
existing DID.
    // The DID is anchored and funded if newly created.
    let (holder_doc, _) = create_or_load_did(
        holder_doc_file,
        holder_fragment_file,
        stronghold_path
    ).await.map_err(|e| {
        eprintln!("Error creating holder DID: {:?}", e);
        StatusCode::INTERNAL_SERVER_ERROR
    })?;

    Ok(Json(DidRespon
        did: holder_d
    }))
}
```

This website uses cookies

We use cookies to improve your experience. You can manage your preferences below. Read our Cookie Policy

Explanation:

- **What it does**: The `ho` `did` endpoint. It calls `create_or_load_did` (a utility function) to either create a

new Decentralized Identifier or load an existing one from `holder_doc.json`. The Decentralized Identifier is anchored to the IOTA network, and the function returns a JSON response containing the Decentralized Identifier string.

- **Why it matters**: This creates the student's identity on the IOTA network, stored securely using Stronghold. The Decentralized Identifier is the foundation for issuing and verifying credentials.
- **Key concepts**:
  - **Decentralized Identifier** Creation: Uses IOTA Identity to generate and publish a Decentralized Identifier document.
  - **Stronghold**: Securely stores the Decentralized Identifier's private keys.
  - **Axum Endpoint**: Handles HTTP requests and returns JSON responses.

The `create_or_load_did` function is a critical helper function used by the `holder_create_did` endpoint to either create a new Decentralized Identifier (DID) for the Student (Holder) or load an existing one from disk.

```
// iota-identity-backend/src/main.rs (create_or_load_did)

// Helper function to create or load a DID document and its signing
fragment
async fn create_or_load_did(
    doc_file: &str,
    fragment_file: &str,
    stronghold_path: &str,
) -> Result<(IotaDocument, String)> {
    let storage =
get_stronghold_storage(Some(PathBuf::from(stronghold_path)))?;

    if !PathBuf::from(doc_file).exists() {
        let client = get_funded_client(&storage).await?;
        let (doc, frag) = create_did_document(&client,
&storage).await?;

        // Save to disk for persistence between server restarts
        fs::write(doc_file, doc.to_json()?)?;
        fs::write(fragment_file, &frag)?;

        println!(">>
        Ok((doc, frag
    } else {
        let doc_json
        let doc = Iot
        let frag =
fs::read_to_string(fr

        println!(">>
        Ok((doc, frag))
```

This website uses cookies

We use cookies to improve your experience. You can manage your preferences below. Read our Cookie Policy

```
    }
}
```

Explanation:

- **What it does**: The `create_or_load_did` function checks if a Decentralized Identifier document exists at the specified `doc_file` path. If it doesn't exist, it creates a new Decentralized Identifier using the IOTA Identity library, funds it via an IOTA client, and saves the Decentralized Identifier document and its signing fragment to `doc_file` and `fragment_file` respectively. If the file exists, it loads the Decentralized Identifier document and fragment from disk. The function returns a tuple containing the `IotaDocument` (the Decentralized Identifier document) and a `String` (the signing fragment).
- **Why it matters**: This function ensures the Student's Decentralized Identifier is either created and anchored to the IOTA network or loaded for reuse, providing persistence across server restarts. It abstracts the complexity of Decentralized Identifier creation and storage, making it reusable across different endpoints (e.g., `holder_create_did`).
- **Key concepts**:
  - **Decentralized Identifier Document**: A JSON object containing the student's decentralized identifier (e.g., `did:iota`:...) and associated data like public keys.
  - **Stronghold Storage**: A secure storage system for private keys, accessed via `get_stronghold_storage`.
  - **Signing Fragment**: A reference to the verification method (e.g., an `Ed25519 key`) used for signing credentials or presentations.
  - **Persistence**: Saves the Decentralized Identifier and fragment to disk to avoid recreating them, ensuring consistency in the identity flow.
  - **IOTA Client**: The `get_funded_client` function provides a funded client to anchor the Decentralized Identifier to the IOTA network.

Context

The `create_or_load_did` function, used by the `holder_create_did` endpoint, relies on three helper fun
functions, defined in `crat
the IOTA Identity framew
management of Decentra

- `get_stronghold_stor`
- `get_funded_client`: (
network operations.
- `create_did_document`: Generates and publishes a Decentralized Identifier document to the IOTA network.

This website uses cookies

We use cookies to improve your experience. You can manage your preferences below. Read our Cookie Policy

Below, we explain each function, including its purpose, functionality, and role in the workshop's identity flow.

Helper Function: `get_stronghold_storage`

This function initializes a Stronghold storage instance for securely managing cryptographic keys.

```
// iota-identity-backend/crates/identity_logic/src/lib.rs
(get_stronghold_storage)

/// Returns a StrongholdStorage instance for persistent key storage.
pub fn get_stronghold_storage(
    path: Option<PathBuf>,
) -> Result<Storage<StrongholdStorage, StrongholdStorage>,
anyhow::Error> {
    let path = path.unwrap_or_else(random_stronghold_path);
    let password = Password::from("secure_password".to_owned());
    let stronghold = StrongholdSecretManager::builder()
        .password(password.clone())
        .build(path.clone())?;
    let stronghold_storage = StrongholdStorage::new(stronghold);
    Ok(Storage::new(
        stronghold_storage.clone(),
        stronghold_storage.clone(),
    ))
}
```

Explanation:

- **What it does**: The `get_stronghold_storage` function creates a `StrongholdStorage` instance, which is used to securely store cryptographic keys. It takes an optional file path (`path`) for the Stronghold file; if none is provided, it generates a random path using `random_stronghold_path`. The function sets a default password (`secure_password`), builds a `StrongholdSecretManager`, and wraps it in a `StrongholdStorage` instance, returning it as a `Stor`
- **Why it matters**: Secu used in Decentralized Verifiable Presentatio protected, preventing storage setup for all i
- **Key concepts**:
  - **Stronghold**: A se

This website uses cookies

We use cookies to improve your experience. You can manage your preferences below. Read our Cookie Policy

- *PathBuf*: A Rust type for handling file paths, used to specify where the Stronghold file is stored.
- **Password**: Secures the Stronghold file to prevent unauthorized access.
- **Storage Type**: Combines key storage (`JwkStorage`) and key ID storage (`KeyIdStorage`) for IOTA Identity compatibility.

Helper Function: `get_funded_client`

This function creates an IOTA Identity client with sufficient funds for network operations.

```
// iota-identity-backend/crates/identity_logic/src/lib.rs
(get_funded_client)

/// Returns an IOTA Identity client funded for publishing operations.
pub async fn get_funded_client<K, I>(
    storage: &Storage<K, I>,
) -> Result<IdentityClient<StorageSigner<K, I>>, anyhow::Error>
where
    K: JwkStorage,
    I: KeyIdStorage,
{
    let generate = storage
        .key_storage()
        .generate(KeyType::new("Ed25519"), JwsAlgorithm::EdDSA)
        .await?;

    let public_key_jwk = generate
        .jwk
        .to_public()
        .expect("public components should be derivable");

    let signer = StorageSigner::new(storage, generate.key_id,
public_key_jwk);
    let sender_address =
IotaAddress::from(&Signer::public_key(&signer).await?);

    request_funds(&se

    let read_only_cli
    let identity_clie
signer).await?;

    Ok(identity_clien
}
```

This website uses cookies

We use cookies to improve your experience. You can manage your preferences below. Read our Cookie Policy

Explanation:

- **What it does**: The `get_funded_client` function generates a new Ed25519 key pair in the provided `storage`, creates a `StorageSigner` for signing transactions, derives an IOTA address from the public key, and requests test funds for that address from the IOTA test network faucet. It then builds an `IdentityClient` by combining a read-only IOTA client (from `get_read_only_client`) with the signer, enabling the client to perform funded operations like publishing Decentralized Identifiers.
- **Why it matters**: Publishing a Decentralized Identifier to the IOTA network requires funds to cover transaction fees. This function automates key generation, funding, and client setup, ensuring the client is ready for identity operations. It's a critical step for creating and anchoring Decentralized Identifiers in the workshop.
- **Key concepts**:
  - **Ed25519**: A secure cryptographic algorithm used for signing transactions and credentials.
  - **StorageSigner**: Combines the storage and key pair to sign IOTA transactions.
  - **IOTA Address**: Derived from the public key, used to request test funds from the faucet.
  - **IdentityClient**: A client that interacts with the IOTA network for identity operations, requiring both a signer and network access.
  - **Test Funds**: Necessary for covering transaction costs on the IOTA test network.

Helper Function: `create_did_document`

This function creates and publishes a Decentralized Identifier document to the IOTA network.

```
// iota-identity-backend/crates/identity_logic/src/lib.rs
(create_did_document)

/// Creates a new IotaDocument, generates a key, publishes the
Decentralized Identifier to the network, and returns the document and
fragment.
pub async fn create_d
    identity_client: 
    storage: &Storage
) -> anyhow::Result<(
where
    K: identity_stora
    I: identity_stora
    S: Signer<IotaKey
{
    let mut unpublished: IotaDocument =
IotaDocument::new(identity_client.network());
```

This website uses cookies

We use cookies to improve your experience. You can manage your preferences below. Read our Cookie Policy

```
    let verification_method_fragment = unpublished
        .generate_method(
            storage,
            JwkMemStore::ED25519_KEY_TYPE,
            JwsAlgorithm::EdDSA,
            None,
            MethodScope::VerificationMethod,
        )
        .await?;

    let document = identity_client
        .publish_did_document(unpublished)
        .with_gas_budget(TEST_GAS_BUDGET)
        .build_and_execute(identity_client)
        .await?
        .output;

    Ok((document, verification_method_fragment))
}
```

Explanation:

- **What it does**: The `create_did_document` function creates a new, unpublished `IotaDocument` for the specified IOTA network, generates an Ed25519 verification method (key pair) stored in `storage`, and adds it to the document. It then publishes the document to the IOTA network using the provided `identity_client`, allocating a gas budget (`TEST_GAS_BUDGET`) for the transaction. The function returns the published Decentralized Identifier document and the fragment (identifier) of the verification method.
- **Why it matters**: This function is the core of Decentralized Identifier creation, anchoring the Student's identity to the IOTA network. The published Decentralized Identifier document and its verification method enable secure signing of credentials and presentations, forming the foundation of the workshop's identity flow.
- **Key concepts**:
  - **IotaDocument**: A
    including its iden
  - **Verification Met**
    a fragment (e.g.,
  - **Publish Decentra**
    document to the
  - **Gas Budget**: Allo
    Decentralized Ide

This website uses cookies

We use cookies to improve your experience. You can manage your preferences below. Read our Cookie Policy

Integration with `create_or_load_did`

These helper functions are called by `create_or_load_did` to create a new Decentralized Identifier:

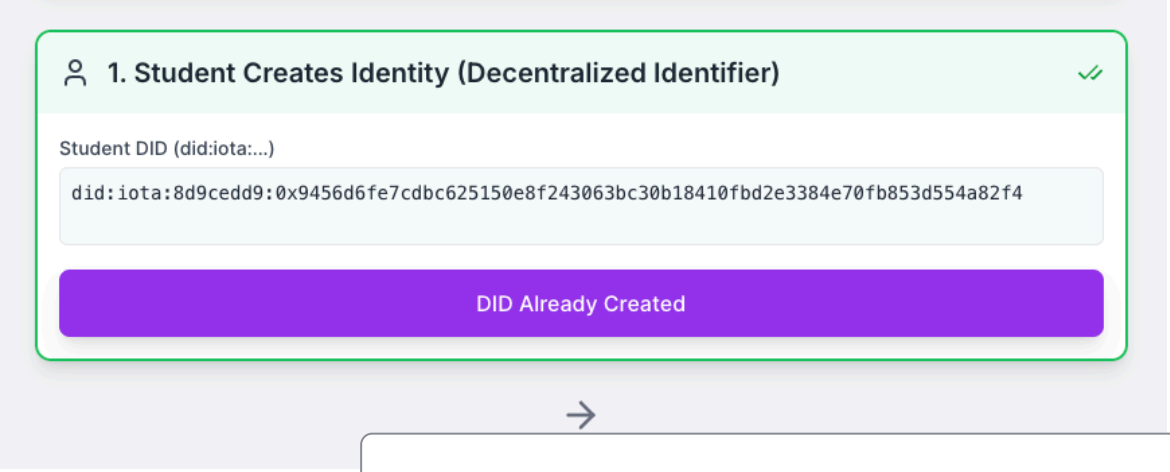
- `get_stronghold_storage`: Provides secure storage for the Decentralized Identifier's private keys.
- `get_funded_client`: Creates a funded IOTA Identity client to interact with the network.
- `create_did_document`: Generates and publishes the Decentralized Identifier document, returning it along with the signing fragment.

Together, they ensure the Student's Decentralized Identifier is created securely, funded, and anchored to the IOTA network, or loaded from disk if it already exists.

Checkpoint

- Verify that `get_stronghold_storage` creates a Stronghold file (e.g., `holder.stronghold`).
- Check that `get_funded_client` successfully requests test funds by monitoring the IOTA test network faucet.
- Confirm that `create_did_document` produces a valid Decentralized Identifier (e.g., `did:iota:...`) by printing the returned `IotaDocument`'s `id`.

Output after a successful creation of the student's Decentralized Identifier.

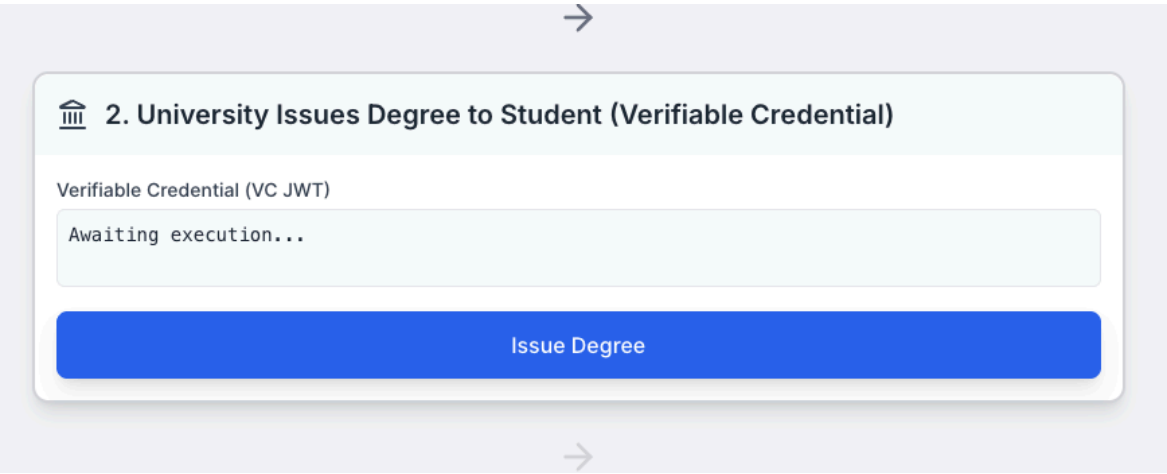


University Issues D

This website uses cookies

We use cookies to improve your experience. You can manage your preferences below. Read our Cookie Policy

Frontend Action			
Button Click (`/issuer/issue-vc`)	`packageId`	`(string)}`	



Frontend (`src/App.jsx`):

```
// src/App.jsx (Issuer VC Handler)

const handleIssueVc = async () => {
    const data = await handleApiCall(2, '/issuer/issue-vc', {});
    if (data && data.jwt) {
        setVcJwt(data.jwt); // Stores the received Verifiable
Credential JWT
    }
};
```

Explanation

- **What it does**: The `handleIssueVc` function sends a POST request to the `/issuer/issue-vc` endpoint with an empty body. The backend generates a Verifiable Credential (University Degree)and returns its JWT, which is stored in the React state (`setVcJwt`) for display and use in the next step.
- **Why it matters**: This triggers the issuance of a degree credential, which the student can later present to employers. It connects the frontend UI to the backend's Verifiable Credential logic.
- **Key concepts**:
  - **Verifiable Credential**: A signed document asserting the student's degree (e.g., Bachelor of
  - **JSON Web Toke** `JSON Web Token`
  - **React State**: Stor

Backend (`src/main.rs`): Decentralized Identifier (D representing a university and returned as a JSON Web Token (JWT) to the React frontend for use in subsequent steps.

This website uses cookies

We use cookies to improve your experience. You can manage your preferences below. Read our Cookie Policy

The `issuer_issue_vc` function handles the `/issuer/issue-vc` endpoint in the Rust Axum backend, orchestrating the creation and signing of the Degree.

```
// Issuer creates DID (if needed) and issues a VC to the Student
async fn issuer_issue_vc(Json(_body): Json<PackageId>) ->
Result<Json<JwtResponse>, StatusCode> {
    let issuer_doc_file = "./issuer_doc.json";
    let issuer_fragment_file = "./issuer_fragment.txt";
    let issuer_stronghold_path = "./issuer.stronghold";

    // 1. Create/Load Issuer DID
    let (issuer_doc, issuer_fragment) = create_or_load_did(
        issuer_doc_file,
        issuer_fragment_file,
        issuer_stronghold_path,
    ).await.map_err(|e| {
        eprintln!("Error creating issuer DID: {:?}", e);
        StatusCode::INTERNAL_SERVER_ERROR
    })?;

    // 2. Load Student DID (must exist from Step 1)
    let holder_doc_json = fs::read_to_string("./holder_doc.json").map_err
StatusCode::NOT_FOUND)?;
    let holder_doc = IotaDocument::from_json(&holder_doc_json).map_err(|_
StatusCode::INTERNAL_SERVER_ERROR)?;

    // 3. Build Verifiable Credential
    let subject = Subject::from_json_value(serde_json::json!({
        "id": holder_doc.id().as_str(),
        "name": "Alice",
        "degree": { "type": "BachelorDegree", "name": "Bachelor of Scienc
Arts" },
        "GPA": "4.0"
    })).map_err(|_| StatusCode::INTERNAL_SERVER_ERROR)?;

    let credential: Credential<Object> = CredentialBuilder::default() //
Annotation Fix
        .id(Url::parse("https://example.edu/credentials/3732").map_err(|_
StatusCode::INTERNAL_
        .issuer(Url::
StatusCode::INTERNAL_
        .type_("Unive
        .subject(subj
        .build().map_

    // 4. Sign Verifi
    let issuer_storag
get_stronghold_storag
StatusCode::INTERNAL_SERVER_ERROR)?;
```

This website uses cookies

We use cookies to improve your experience. You can manage your preferences below. Read our Cookie Policy

```
    let credential_jwt = issuer_doc
        .create_credential_jwt(
            &credential,
            &issuer_storage,
            &issuer_fragment,
            &JwsSignatureOptions::default(),
            None,
        )
        .await
        .map_err(|e| {
            eprintln!("Error signing VC: {:?}", e);
            StatusCode::INTERNAL_SERVER_ERROR
        })?;

    // Return VC JWT string
    Ok(Json(JwtResponse {
        jwt: credential_jwt.as_str().to_string(),
    }))
}
```

Explanation:

- **What it does**: The `issuer_issue_vc` function handles the `/issuer/issue-vc` HTTP endpoint, triggered by the React frontend. It performs four key steps:
  i. **Create/Load Issuer Decentralized Identifier**: Calls `create_or_load_did` to either create a new Decentralized Identifier for the University (issuer) or load an existing one from disk.
  ii. **Load Student Decentralized Identifier**: Reads the Student's Decentralized Identifier document from `holder_doc.json` (created in Step 1).
  iii. **Build Verifiable Credential**: Constructs a Verifiable Credential (Degree) with the Student's Decentralized Identifier, student details (e.g., name, degree, GPA), and an university identifier.
  iv. **Sign Verifiable Credential**: Uses the University's private key (stored in Stronghold) to sign the Verifiable Credential (Degree) as a JSON Web Token, which is returned to the frontend.
- **Why it matters**: This
  to the Student's Dece
  employers in later ste
  Credential" button to
  cryptographically sig
- **Key concepts**:
  - **Verifiable Crede**
    (e.g., Alice's degr

This website uses cookies

We use cookies to improve your experience. You can manage your preferences below. Read our Cookie Policy

- **Decentralized Identifier (DID)**: The Universities's and Student's decentralized identifiers, used to establish trust and ownership.
- **JWT**: A JSON Web Token encoding the Verifiable Credential, signed with the Universities's private key for authenticity.
- **Stronghold**: Securely stores the Universities's private key, accessed via `get_stronghold_storage`.
- **CredentialBuilder**: An IOTA Identity utility for constructing VCs with standardized fields (e.g., `id`, `issuer`, `subject`).

Step-by-Step Breakdown

1. Create/Load University Decentralized Identifier (DID):

- Code:

```
let (issuer_doc, issuer_fragment) = create_or_load_did(
    issuer_doc_file,
    issuer_fragment_file,
    issuer_stronghold_path,
).await.map_err(...)?;
```

- **What it does**: Calls the `create_or_load_did` helper function to either create a new Decentralized Identifier for the University (saved to `issuer_doc.json` and `issuer_fragment.txt`) or load an existing one. The Decentralized Identifier is anchored to the IOTA network if newly created.
- **Why it matters**: The University's Decentralized Identifier establishes the university's identity, used to sign the Verifiable Credential and prove its authenticity.
- **Key concepts**:
  - **Decentralized Identifier Document**: Contains the University's identifier (e.g., `did:iota:...`) and verification methods.
  - **Fragment**: Identifies the signing key (e.g., `#sign-0`) in the Decentralized Identifier document.

2. Load Student D

- **Code**:

```
let holder_doc_json =
fs::read_to_string(".
StatusCode::NOT_FOUND
let holder_doc = Iota
StatusCode::INTERNAL_SERVER_ERROR)?;
```

This website uses cookies

We use cookies to improve your experience. You can manage your preferences below. Read our Cookie Policy

- **What it does**: Reads the Student's Decentralized Identifier document from `holder_doc.json` (created in Step 1) and deserializes it into an `IotaDocument`.
- **Why it matters**: The Student's Decentralized Identifier is needed to link the Verifiable Credential to the student (Alice), ensuring the credential is tied to her identity.
- **Key concepts**:
  - File I/O: Uses Rust's `fs` module to read the Decentralized Identifier document from disk.
  - JSON Deserialization: Converts the JSON string into an `IotaDocument` using `from_json`.

3. Build Verifiable Credential:

- Code:

```
let subject = Subject::from_json_value(serde_json::json!({
    "id": holder_doc.id().as_str(),
    "name": "Alice",
    "degree": { "type": "BachelorDegree", "name": "Bachelor of Science
and Arts" },
    "GPA": "4.0"
})).map_err(...)?;
let credential: Credential<Object> = CredentialBuilder::default()

.id(Url::parse("https://example.edu/credentials/3732").map_err(...)?)
    .issuer(Url::parse(issuer_doc.id().as_str()).map_err(...)?)
    .type_("UniversityDegreeCredential")
    .subject(subject)
    .build().map_err(...)?;
```

- **What it does**: Creates a `Subject` object with the Student's Decentralized Identifier, name, degree details, and GPA. Uses `CredentialBuilder` to construct a Verifiable Credential with a unique ID, the University's Decentralized Identifier, a type (`UniversityDegreeCredential`), and the subject data.
- **Why it matters**: The V
containing verifiable
the W3C Verifiable Cr
- **Key concepts**:
  - **Subject**: The enti
Decentralized Ide
  - **CredentialBuilde**
standardized fiel
  - **JSON Macro**: The `serde_json::json!` macro creates a JSON object for the subject data.

This website uses cookies

We use cookies to improve your experience. You can manage your preferences below. Read our Cookie Policy

4. Sign Verifiable Credential:

- Code:

```
let issuer_storage =
get_stronghold_storage(Some(PathBuf::from(issuer_stronghold_path))).map_e
let credential_jwt = issuer_doc
    .create_credential_jwt(
        &credential,
        &issuer_storage,
        &issuer_fragment,
        &JwsSignatureOptions::default(),
        None,
    )
    .await
    .map_err(...)?;
```

- **What it does**: Initializes Stronghold storage for the University's private keys, then uses the `issuer_doc` to sign the Verifiable Credential with the specified `issuer_fragment` (signing key). The signed Verifiable Credential is encoded as a JWT.
- **Why it matters**: Signing the Verifiable Credential ensures its authenticity and integrity, allowing employers (e.g., verifier) to trust that it was issued by the university.
- **Key concepts**:
  - **Digital Signature**: Uses the University's private key (via Stronghold) to sign the Verifiable Credential.
  - **JWT**: Encodes the Verifiable Credential for secure transmission to the frontend.
  - **JwsSignatureOptions**: Configures signing options (e.g., default algorithm is Ed25519).

5. Return Verifiable Credential JWT:

- Code:

```
Ok(Json(JwtResponse {
    jwt: credential_j
}))
```

- **What it does**: Wraps
  and returns it as a JS
- **Why it matters**: The JWT is sent to the React frontend, where it's stored for use in the next step (Verifiable Presentation creation).

This website uses cookies

We use cookies to improve your experience. You can manage your preferences below. Read our Cookie Policy

- **Key concepts**:
  - **Axum Response**: Uses `Json` to serialize the response for HTTP communication.
  - **JwtResponse**: A custom struct to structure the API response.

Integration with the Workshop

The `issuer_issue_vc` function is triggered by the React frontend's "Execute Step 2" button, which calls the `/issuer/issue-vc` endpoint. It relies on:

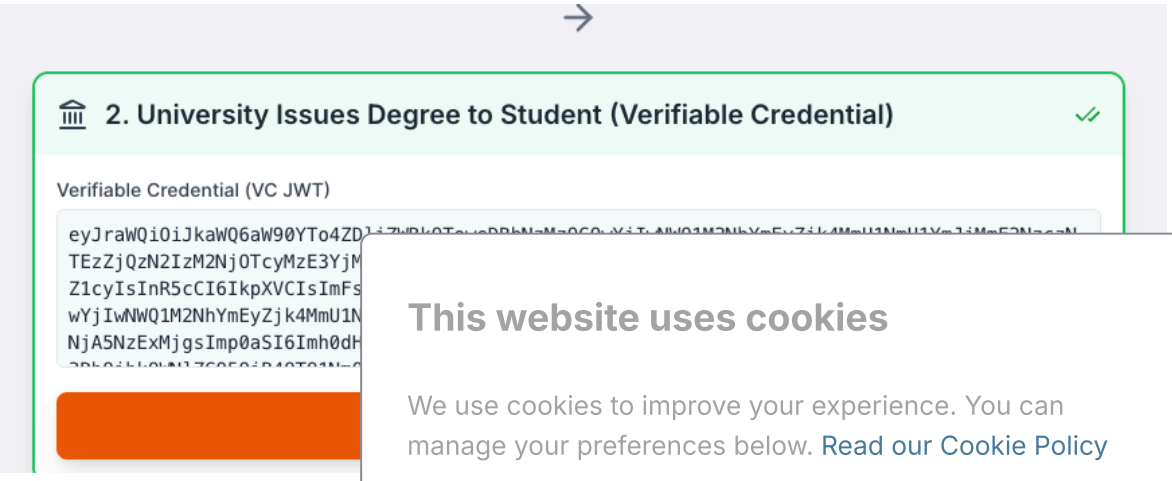
- `create_or_load_did`: To obtain the University's Decentralized Identifier and signing fragment.
- `get_stronghold_storage`: To access the University's private keys for signing.
- The Student's Decentralized Identifier document (`holder_doc.json`) from Step 1.

The returned Verifiable Credential JWT is stored in the frontend's state and used in Step 3 to create a Verifiable Presentation (VP).

Checkpoint

- Run the backend (`cargo run`) and use the frontend to trigger Step 2.
- Verify that `issuer_doc.json`, `issuer_fragment.txt`, and `vc.jwt` are created.
- Check the frontend UI to confirm the VC JWT is displayed (e.g., a long string starting with `eyJ...`).
- Use an online JWT decoder to inspect the Verifiable Credential contents (e.g., Alice's name, degree).

Output after a successful creation of the student's Verifiable Credential.

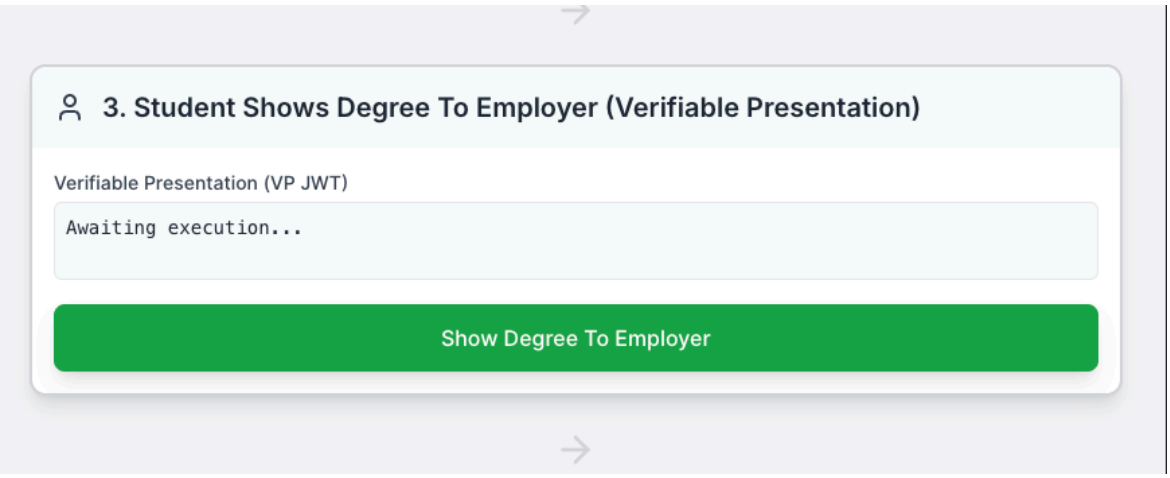


This website uses cookies

We use cookies to improve your experience. You can manage your preferences below. Read our Cookie Policy

Student Shows De

Frontend Action	Rust Endpoint	Payload Sent (Body)	Result Stored
Button Click (`/holder/create-vp`)	`packageId, vcJwt`	`{ "vpJwt string}`	



Frontend (`src/App.jsx`): The client sends the received Verifiable Credential back to the server, asking the server to wrap and sign it.

```
// src/App.jsx (Holder VP Handler)

const handleCreateVp = async () => {
    // FIX: The payload is constructed to send { vcJwt: value } to
match the Rust struct's casing.
    const data = await handleApiCall(3, '/holder/create-vp', { vcJwt
});
    if (data && data.jwt) {
        setVpJwt(data.jwt); // Stores the Verifiable Presentation JWT
    }
};
```

Explanation:

- **What it does**: The ha
  `/holder/create-vp` e
  from the previous ste
  which is stored in the
- **Why it matters**: This
  Presentation, proving
  an employer.
- **Key concepts**:
  - **Verifiable Presentation**: A signed package containing the Verifiable Credential, proving the Student's ownership.

This website uses cookies

We use cookies to improve your experience. You can manage your preferences below. Read our Cookie Policy

- **Payload**: Sends the Verifiable Credential JWT to the backend for inclusion in the Verifiable Presentation.
- **React State**: Stores the Verifiable Presentation JWT for the final verification step.

Backend (`src/main.rs`): In this step, the Holder (student, Alice) packages the received Verifiable Credential (VC) into a Verifiable Presentation (VP) and signs it for presentation to a employer (verifier). The Verifiable Presentation proves that Alice controls the Verifiable Credential and hasn't tampered with it.

Backend: `holder_create_vp` Function

The `holder_create_vp` function handles the `/holder/create-vp` endpoint in the Rust Axum backend, receiving the Verifiable Credential JWT from the frontend and creating a signed Verifiable Presentation.

```
// iota-identity-backend/src/main.rs (holder_create_vp)

// Step 3: Holder creates a Verifiable Presentation (VP)
async fn holder_create_vp(Json(body): Json<VcJwt>) ->
Result<Json<JwtResponse>, StatusCode> {
    let stronghold_path = "./holder.stronghold";
    let holder_doc_file = "./holder_doc.json";
    let holder_fragment_file = "./holder_fragment.txt";

    // 1. Load Holder DID
    let (holder_doc, holder_fragment) = create_or_load_did(
        holder_doc_file,
        holder_fragment_file,
        stronghold_path,
    ).await.map_err(|_| StatusCode::NOT_FOUND)?;

    let holder_storage =
get_stronghold_storage(Some(PathBuf::from(stronghold_path))).map_err(|_|
StatusCode::INTERNAL_SERVER_ERROR)?;

    let challenge = "challenge-123";
    let expires =
Timestamp::now_utc().

    // 2. Build VP
    let presentation:
Type Annotation Fix
        holder_doc.id
        Default::defa
        .credential(J
        .build().map_err(|e| {
            eprintln!("Error building presentation: {:?}", e);
```

This website uses cookies

We use cookies to improve your experience. You can manage your preferences below. Read our Cookie Policy

```
            StatusCode::INTERNAL_SERVER_ERROR
        })?;

    // 3. Sign VP
    let vp_jwt: Jwt = holder_doc
        .create_presentation_jwt(
            &presentation,
            &holder_storage,
            &holder_fragment,
            &JwsSignatureOptions::default().nonce(challenge.to_owned()),

&JwtPresentationOptions::default().expiration_date(exespires),
        )
        .await
        .map_err(|e| {
            eprintln!("Error signing VP: {:?}", e);
            StatusCode::INTERNAL_SERVER_ERROR
        })?;

    // Return VP JWT string
    Ok(Json(JwtResponse {
        jwt: vp_jwt.as_str().to_string(),
    }))
}
```

Explanation:

- What it does: The `holder_create_vp` function handles the `/holder/create-vp` HTTP endpoint, triggered by the React frontend. It performs three key steps:
  i. **Load Holder Decentralized Identifier**: Retrieves the Student's Decentralized Identifier document and signing fragment from disk (created in Step 1).
  ii. **Build Verifiable Presentation**: Creates a Verifiable Presentation that embeds the received Verifiable Credential JWT (`body.vc_jwt`), sets the Student's Decentralized Identifier as the presenter, and configures expiration and challenge parameters.
  iii. **Sign Verifiable P**
  Stronghold) to sig
  proves Alice cont
- **Why it matters**: This
  presentation of their
  employers. The Verifi
  Verifiable Credential
  Student's role in the i
- **Key concepts**:

This website uses cookies

We use cookies to improve your experience. You can manage your preferences below. Read our Cookie Policy

- **Verifiable Presentation (VP)**: A signed package containing one or more Verifiable Credentials, proving the presenter's control and ownership.
- **Challenge/Nonce**: A unique string (`challenge-123`) that prevents replay attacks and ensures the VP is fresh.
- **Expiration**: The VP expires after 10 minutes, adding security by limiting its validity period.
- **PresentationBuilder**: An IOTA Identity utility for constructing VPs with the presenter's Decentralized Identifier and embedded credentials.

Step-by-Step Breakdown

1. Load Holder Decentralized Identifier:
   - Code:

```
let (holder_doc, holder_fragment) = create_or_load_did(
    holder_doc_file,
    holder_fragment_file,
    stronghold_path,
).await.map_err(|_| StatusCode::NOT_FOUND)?;
```

- **What it does**: Calls the `create_or_load_did` helper function to load the Student's Decentralized Identifier document and signing fragment from `holder_doc.json` and `holder_fragment.txt` (created in Step 1). Since the Decentralized Identifier already exists, it reads from disk rather than creating a new one.
- **Why it matters**: The Student's Decentralized Identifier is needed to identify Alice as the presenter of the Verifiable Presentation and to access her private key for signing. This ensures the Verifiable Presentation is linked to her identity.
- **Key concepts**:
  - **Decentralized Identifier Persistence**: The Decentralized Identifier document and fragment are loaded from disk, ensuring consistency across API calls.
  - **HTTP Error Handling**: Returns `StatusCode::NOT_FOUND` if the Decentralized Ide
    completed.

2. Initialize Storage and
   - Code:

```
let holder_storage =
get_stronghold_storag
StatusCode::INTERNAL_SERVER_ERROR)?;
```

This website uses cookies

We use cookies to improve your experience. You can manage your preferences below. Read our Cookie Policy

```
let challenge = "challenge-123";
let expires =
Timestamp::now_utc().checked_add(Duration::minutes(10)).unwrap();
```

- **What it does**: Initializes Stronghold storage for accessing the Student's private keys and sets security parameters: a fixed challenge (`challenge-123`) for nonce validation and an expiration time 10 minutes from now.
- **Why it matters**: Stronghold provides secure access to Alice's private key for signing the Verifiable Presentation. The challenge prevents replay attacks (using the same Verifiable Presentation multiple times), and the expiration ensures the Verifiable Presentation has limited validity, enhancing security.
- **Key concepts**:
  - **Stronghold Storage**: Securely stores and retrieves the Student's private keys.
  - **Challenge/Nonce**: A unique identifier included in the Verifiable Presentation signature, verified by the recipient to ensure freshness.
  - **Timestamp & Duration**: Sets the Verifiable Presentation's expiration using UTC time, preventing indefinite use.

3. Build Verifiable Presentation:

   - Code:

```
let presentation: Presentation<Jwt> = PresentationBuilder::new(
    holder_doc.id().to_url().into(),
    Default::default())
    .credential(Jwt::new(body.vc_jwt))
    .build().map_err(|e| {
        eprintln!("Error building presentation: {:?}", e);
        StatusCode::INTERNAL_SERVER_ERROR
    })?;
```

- **What it does**: Creates a `PresentationBuilder` with the Student's Decentralized Identifier as the presenter, embeds the received Verifiable Credential JWT (`body.vc_jwt`) from the frontend, and builds the presentation structure. The `Jwt::n` a credential within the
- **Why it matters**: This contains Alice's degr format allows employ and Alice's ownership
- **Key concepts**:
  - **PresentationBuil** presenter's Decentralized Identifier and embedded credentials.

This website uses cookies

We use cookies to improve your experience. You can manage your preferences below. Read our Cookie Policy

- **Credential Embedding**: The VC JWT (from Step 2) is included in the Verifiable Presentation for presentation to employers.
- **URL Conversion**: The Student's Decentralized Identifier is converted to a URL format for use in the presentation.

4. Sign Verifiable Presentation:

   - Code:

```
let vp_jwt: Jwt = holder_doc
    .create_presentation_jwt(
        &presentation,
        &holder_storage,
        &holder_fragment,
        &JwsSignatureOptions::default().nonce(challenge.to_owned()),
        &JwtPresentationOptions::default().expiration_date(expires),
    )
    .await
    .map_err(|e| {
        eprintln!("Error signing VP: {:?}", e);
        StatusCode::INTERNAL_SERVER_ERROR
    })?;
```

- **What it does**: Uses the Student's Decentralized Identifier document to sign the presentation with her private key (accessed via `holder_storage` and identified by `holder_fragment`). The signature includes the challenge as a nonce and sets the expiration time. The signed presentation is encoded as a JWT.
- **Why it matters**: Signing the VP with Alice's private key proves she controls the Decentralized Identifier and the embedded Verifiable Credential, enabling employers to trust that she's the legitimate owner of the degree credential.
- **Key concepts**:
  - **Digital Signature**: Uses the Student's private key to create a cryptographic proof of ownership.
  - **create_presentation_jwt**: IOTA Identity function that signs the presentation and encodes it as a JWT.
  - **JwsSignatureOp**
    for replay protect
  - **JwtPresentation**
    expiration time.

5. Return Verifiable Pres

   - Code:

This website uses cookies

We use cookies to improve your experience. You can manage your preferences below. Read our Cookie Policy

```
Ok(Json(JwtResponse {
    jwt: vp_jwt.as_str().to_string(),
}))
```

- **What it does**: Wraps the signed VP JWT in a `JwtResponse` struct and returns it as a JSON response to the React frontend.
- **Why it matters**: The VP JWT is sent to the frontend, where it's stored for use in Step 4 (verification). This completes the Student's role in creating a presentable credential package.
- **Key concepts**:
  - **Axum JSON Response**: Serializes the VP JWT for HTTP communication with the frontend.
  - **JwtResponse**: Custom API response structure containing the signed VP.

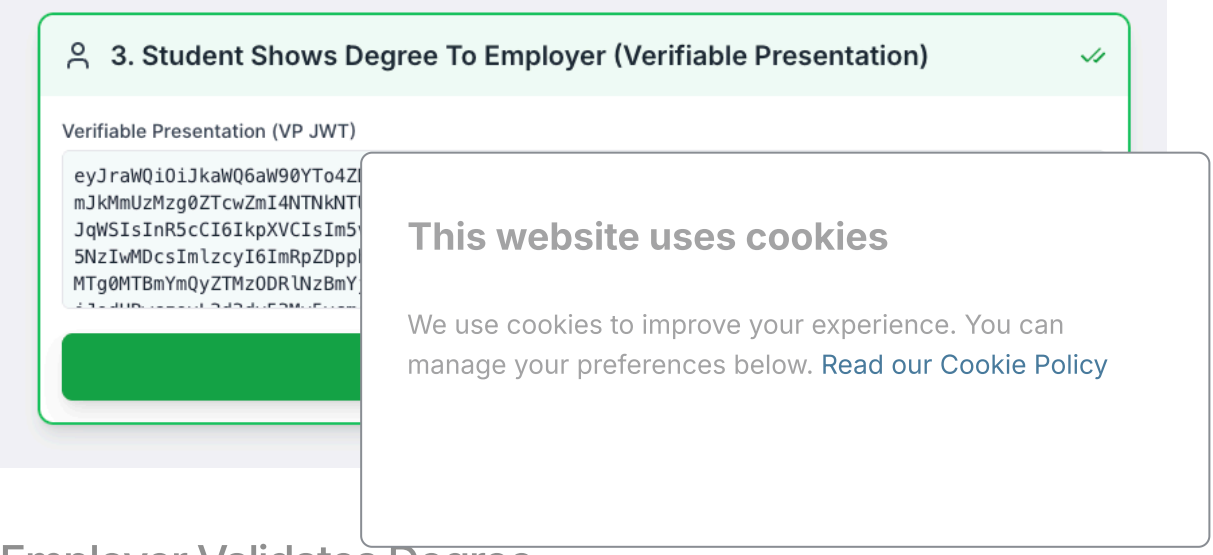
Integration with the Workshop

The `holder_create_Verifiable Presentation` function is triggered by the React frontend's "Execute Step 3" button, which calls the `/holder/create-vp` endpoint with the Verifiable Credential JWT from Step 2. It relies on:

- Input: The Verifiable Credential JWT (`body.vc_jwt`) received from the frontend (issued in Step 2).
- **Dependencies**: `create_or_load_did` to load the Student's Decentralized Identifier and `get_stronghold_storage` to access private keys.
- **Prerequisites**: The Student's Decentralized Identifier must exist from Step 1 (`holder_doc.json`).

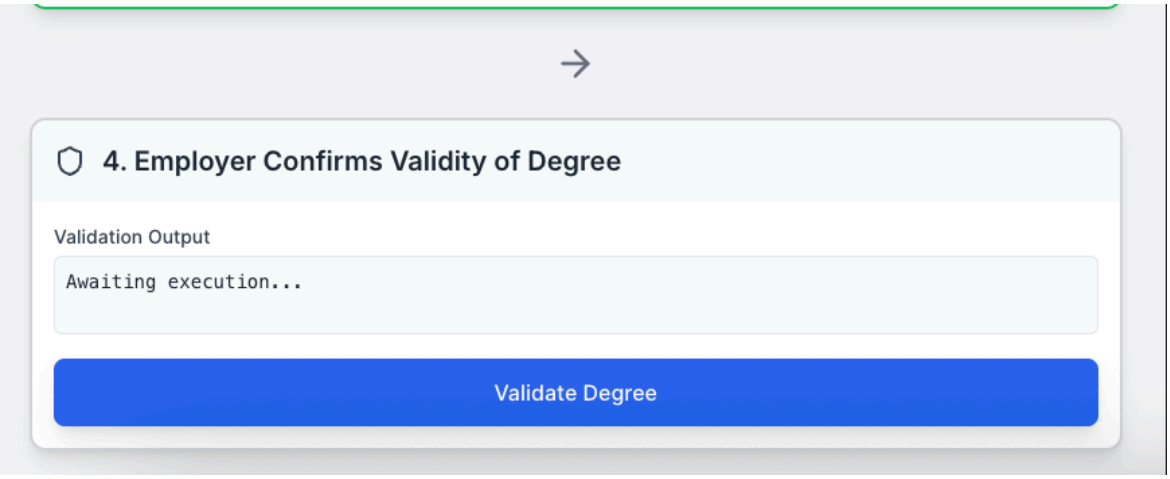
The returned VP JWT is stored in the frontend's state and used in Step 4 for verification by the employer.

Output after a successful creation of the student's Verifiable Presentation.



Employer Validates Degree

Frontend Action	Rust Endpoint	Payload Sent (Body)	Result Stored
Button Click (`/verifier/validate`)	`packageId, vpJwt`	`{ "validationOutput log}`	



Frontend (`src/App.jsx`):

```
// src/App.jsx (Verifier Validation Handler)

const handleValidateVp = async () => {
    // Sends the VP JWT to the server for verification.
    const data = await handleApiCall(4, '/verifier/validate', { vpJwt
});
    if (data) {
        setValidationOutput(data.output);
        setIsValidationSuccess(data.success); // Updates UI to show
SUCCESS/FAIL
    }
};
```

Explanation:

- What it does: The `har`
  `/verifier/validate`
  a validation result (log
  to display the outcom
- Why it matters: This c
  authenticity, ensuring
- Key concepts:
  - Validation Result: Displays whether the Verifiable Presentation and Verifiable Credential are valid.

This website uses cookies

We use cookies to improve your experience. You can manage your preferences below. Read our Cookie Policy

- React State: Updates the UI with the verification log and success status.
- API Call: Sends the VP JWT to the backend for network-based verification.

Backend (`src/main.rs`): This is the final, network-intensive verification step.

In this step, the Verifier (employer) validates the Verifiable Presentation (VP) and its embedded Verifiable Credential (VC) received from the Holder (student, Alice). This ensures the degree credential is authentic, untampered, and issued by a trusted university, completing the IOTA Identity flow.

Backend: `verifier_validate` Function

The `verifier_validate` function handles the `/verifier/validate` endpoint in the Rust Axum backend, receiving the VP JWT from the frontend and performing a multi-step validation process.

```
// Step 4: Employer validates the Verifiable Presentation and embedded Ve
Credential
async fn verifier_validate(Json(body): Json<VpJwt>) ->
Result<Json<ValidationResponse>, StatusCode> {
    let vp_jwt = Jwt::new(body.vp_jwt);
    let challenge = "challenge-123";
    let mut output = String::new();

    // 1. Setup Resolver and Read-Only Client
    let verifier_client = get_read_only_client().await.map_err(|_|
StatusCode::INTERNAL_SERVER_ERROR)?;
    let mut resolver: Resolver<IotaDocument> = Resolver::new();
    resolver.attach_iota_handler(verifier_client);

    let presentation_verifier_options =
JwsVerificationOptions::default().nonce(challenge.to_owned());
    output.push_str(&format!("🔍 Using challenge: {}\n", challenge));

    // 2. Resolve Holder DID
    let holder_did: CoreDID = JwtPresentationValidatorUtils::extract_holc
        .map_err(|_|
    output.push_str(&

    let holder_doc: I
        output.push_s
        StatusCode::I
    })?;
    output.push_str(&
network: {}\n", holde

    // 3. Validate VP Signature and Challenge
```

This website uses cookies

We use cookies to improve your experience. You can manage your preferences below. Read our Cookie Policy

```
    let vp_validation_options = JwtPresentationValidationOptions::default
        .presentation_verifier_options(presentation_verifier_options);

    let decoded_vp: DecodedJwtPresentation<Jwt> =
JwtPresentationValidator::with_signature_verifier(EdDSAJwsVerifier::defau
        .validate(&vp_jwt, &holder_doc, &vp_validation_options)
        .map_err(|e| {
            output.push_str(&format!(" VP Validation Failed: {:?}\n", e))
            StatusCode::BAD_REQUEST
        })?;

    output.push_str(" VP JWT verified successfully\n");
    output.push_str(&format!(" VP Holder matches Subject: {}\n",
decoded_vp.presentation.holder));

    // 4. Extract and Validate Embedded Credentials
    let jwt_credentials: &Vec<Jwt> = &decoded_vp.presentation.verifiable_
    output.push_str(&format!(" 📦 VP contains {} credential(s)\n",
jwt_credentials.len()));

    let issuers: Vec<CoreDID> = jwt_credentials
        .iter()
        .map(|jwt|
identity_iota::credential::JwtCredentialValidatorUtils::extract_issuer_fr
        .collect::<Result<Vec<CoreDID>, _>>()
        .map_err(|_| StatusCode::INTERNAL_SERVER_ERROR)?;

    let issuers_documents: HashMap<CoreDID, IotaDocument> =
resolver.resolve_multiple(&issuers).await
        .map_err(|_| StatusCode::INTERNAL_SERVER_ERROR)?;

    let credential_validator =
JwtCredentialValidator::with_signature_verifier(EdDSAJwsVerifier::default
    let credential_validation_options = JwtCredentialValidationOptions::d
        .subject_holder_relationship(holder_did.to_url().into(),
SubjectHolderRelationship::AlwaysSubject);

    for (index, jwt_vc) in jwt_credentials.iter().enumerate() {
        let issuer_doc = &issuers_documents[&issuers[index]];
        let result: R
            .validate
FailFast::FirstError)

        match result
            Ok(decode
                let
                .
                .
                .
                .and_then(|subj| subj.id.as_ref())
```

This website uses cookies

We use cookies to improve your experience. You can manage your preferences below. Read our Cookie Policy

```
                    .map(|id| id.as_str())
                    .unwrap_or("unknown");

                output.push_str(&format!(
                    "✅ Credential [{}]: verified successfully, subject:
                    index + 1,
                    subject_id
                ));
            },
            Err(e) => {
                output.push_str(&format!("Credential [{}] Validation Fail
index + 1, e));
                return Ok(Json(ValidationResponse { success: false, outpu
            }
        }
    }

    output.push_str("\n All credentials in the VP are valid!");
    Ok(Json(ValidationResponse { success: true, output }))
}
```

Explanation:

- What it does: The verifier_validate function handles the /verifier/validate HTTP endpoint, triggered by the React frontend. It validates the Verifiable Presentation JWT received from the frontend (body.Verifiable Presentation_jwt) by performing four key steps:
  i. Setup Resolver and Client: Initializes an IOTA resolver and read-only client to fetch Decentralized Identifier documents from the IOTA network.
  ii. Resolve Student Decentralized Identifier: Extracts and resolves the Student's Decentralized Identifier from the Verifiable Presentation to retrieve their public key.
  iii. Validate Verifiable Presentation Signature and Challenge: Verifies the Verifiable Presentation signature and nonce (challenge) to ensure its authenticity and freshness.
  iv. Validate Embedded Credentials: Extracts and verifies the signatures of all embedded Verifi
  universities.
- Why it matters: This f
  employer (Verifier) to
  (Verifiable Credential
  without intermediarie
  decentralized degree
- Key concepts:

This website uses cookies

We use cookies to improve your experience. You can manage your preferences below. Read our Cookie Policy

- Resolver: A tool to fetch Decentralized Identifier documents from the IOTA network, providing public keys for signature verification.
- Verifiable Presentation Validation: Verifies the Verifiable Presentation signature, nonce, and Student's identity.
- Verifiable Credential Validation: Confirms the embedded Verifiable Credential signature and university, ensuring it's untampered and trustworthy.
- Nonce/Challenge: Ensures the Verifiable Presentation is fresh and not reused (challenge-123).
- Validation Log: A detailed string (output) tracking each step's success or failure, returned to the frontend.

Step-by-Step Breakdown

1. Setup Resolver and Read-Only Client:
   - Code:

```
let verifier_client = get_read_only_client().await.map_err(|_|
StatusCode::INTERNAL_SERVER_ERROR)?;
let mut resolver: Resolver<IotaDocument> = Resolver::new();
resolver.attach_iota_handler(verifier_client);
let presentation_verifier_options =
JwsVerificationOptions::default().nonce(challenge.to_owned());
output.push_str(&format!(" Using challenge: {}\n", challenge));
```

- What it does: Initializes a read-only IOTA client (via get_read_only_client) and sets up a Resolver to fetch Decentralized Identifier documents from the IOTA network. Configures verification options with the expected challenge (challenge-123) and logs it to the output string.
- Why it matters: The resolver is essential for retrieving public keys from the IOTA network to verify signatures. The read-only client ensures network access without requiring funds.
- Key concepts:
  - Resolver: Fetches Decentralized Identifier documents using the IOTA network.
  - Read-Only Client
    Identifiers withou
  - Challenge: Ensur
    expected value (

2. Resolve Holder Dece
   - Code:

This website uses cookies

We use cookies to improve your experience. You can manage your preferences below. Read our Cookie Policy

```
let holder_did: CoreDID =
JwtPresentationValidatorUtils::extract_holder(&vp_jwt)
    .map_err(|_| StatusCode::BAD_REQUEST)?;
output.push_str(&format!("Extracted holder DID from VP: {}\n",
holder_did));
let holder_doc: IotaDocument =
resolver.resolve(&holder_did).await.map_err(|e| {
    output.push_str(&format!("Failed to resolve holder DID: {:?}\n",
e));
    StatusCode::INTERNAL_SERVER_ERROR
})?;
output.push_str(&format!("✅ Resolved holder DID from network: {}\n",
holder_doc.id()));
```

- What it does: Extracts the Student's Decentralized Identifier from the VP JWT and uses the resolver to fetch the corresponding Decentralized Identifier document from the IOTA network. Logs the extracted and resolved Decentralized Identifier to the output string.
- Why it matters: The Student's Decentralized Identifier document contains Alice's public key, needed to verify the Verifiable Presentation signature. Resolving it from the network ensures the Decentralized Identifier is valid and untampered.
- Key concepts:
  - CoreDID: A standardized Decentralized Identifier representation (e.g., did:iota:...).
  - Resolver: Retrieves the Decentralized Identifier document, including public keys, from the IOTA network.
  - Logging: Tracks the Decentralized Identifier resolution process for debugging and user feedback.

3. Validate Verifiable Presentation Signature and Challenge:

   - Code:

This website uses cookies

We use cookies to improve your experience. You can manage your preferences below. Read our Cookie Policy

```
let vp_validation_options = JwtPresentationValidationOptions::default()
    .presentation_verifier_options(presentation_verifier_options);
let decoded_vp: DecodedJwtPresentation<Jwt> =
JwtPresentationValidator::with_signature_verifier(EdDSAJwsVerifier::defau
    .validate(&vp_jwt, &holder_doc, &vp_validation_options)
    .map_err(|e| {
        output.push_str(&format!(" VP Validation Failed: {:?}\n", e));
        StatusCode::BAD_REQUEST
    })?;
output.push_str("VP JWT verified successfully\n");
output.push_str(&format!(" VP Student matches Subject: {}\n",
decoded_vp.presentation.holder));
```

- What it does: Configures Verifiable Presentation validation options with the expected nonce, then uses JwtPresentationValidator with an Ed25519 verifier to check the VP's signature and nonce against the Student's public key (from holder_doc). Logs the successful validation and confirms the Student's Decentralized Identifier matches the VP's presenter.
- Why it matters: This verifies that Alice signed the Verifiable Presentation and that it hasn't been tampered with, ensuring she controls the presented credentials. The nonce check prevents replay attacks.
- Key concepts:
  - Signature Verification: Uses the Student's public key to verify the Verifiable Presentation Ed25519 signature.
  - Nonce Validation: Ensures the VP's nonce matches challenge-123.
  - DecodedJwtPresentation: The validated Verifiable Presentation structure, containing the embedded Verifiable Credential and Student's Decentralized Identifier.

4. Extract and Validate Embedded Credentials:
   - Code:

```
let jwt_credentials: &Vec<Jwt> = &decoded_vp.presentation.verifiable_crec
output.push_str(&format!("📦 VP contains {} credential(s)\n",
jwt_credentials.len()));
let issuers: Vec<Core
    .iter()
    .map(|jwt|
identity_iota::creden
    .collect::<Result
    .map_err(|_| Stat
let issuers_documents
resolver.resolve_mult
    .map_err(|_| Stat
let credential_validator =
JwtCredentialValidator::with_signature_verifier(EdDSAJwsVerifier::default
```

This website uses cookies

We use cookies to improve your experience. You can manage your preferences below. Read our Cookie Policy

```
let credential_validation_options = JwtCredentialValidationOptions::defau
    .subject_holder_relationship(holder_did.to_url().into(),
SubjectHolderRelationship::AlwaysSubject);
for (index, jwt_vc) in jwt_credentials.iter().enumerate() {
    let issuer_doc = &issuers_documents[&issuers[index]];
    let result: Result<DecodedJwtCredential<Object>, _> = credential_vali
        .validate(jwt_vc, issuer_doc, &credential_validation_options,
FailFast::FirstError);
    match result {
        Ok(decoded_credential) => {
            let subject_id = decoded_credential
                .credential
                .credential_subject
                .first()
                .and_then(|subj| subj.id.as_ref())
                .map(|id| id.as_str())
                .unwrap_or("unknown");
            output.push_str(&format!(
                "✅ Credential [{}]: verified successfully, subject: {}\
                index + 1,
                subject_id
            ));
        },
        Err(e) => {
            output.push_str(&format!(" Credential [{}] Validation Failed:
index + 1, e));
            return Ok(Json(ValidationResponse { success: false, output })
        }
    }
}
```

- What it does: Extracts the Verifiable Credential from the Verifiable Presentation, retrieves the University's Decentralized Identifier for each Verifiable Credential, and resolves their Decentralized Identifier documents. Validates each Verifiable Credential's signature against the University's public key, ensuring the Verifiable Credential was issued by the university and the subject matches the Student's Decentralized Identifier. Logs each credential's validation result.
- Why it matters: This c
  was issued by a trust
  holder relationship ch
  Verifiable Credential.
- Key concepts:
  - Credential valida
    University's publi
  - University Resolu
    document to access their public key.

This website uses cookies

We use cookies to improve your experience. You can manage your preferences below. Read our Cookie Policy

    - Subject-Holder Relationship: Ensures the Verifiable Credential subject (Alice) matches the Student's Decentralized Identifier.
    - FailFast: Stops validation on the first error to avoid unnecessary processing.

5. Return Validation Result:
    - Code:

```
output.push_str("\n🎉 All credentials in the VP are valid!");
Ok(Json(ValidationResponse { success: true, output }))
```

- What it does: If all validations pass, logs a success message and returns a ValidationResponse with success: true and the detailed validation log (output) to the frontend.
- Why it matters: The response informs the frontend whether the Verifiable Presentation and its Verifiable Credential are valid, updating the UI to show “SUCCESS” or “FAIL” and displaying the validation log.
- Key concepts:
    - ValidationResponse: A custom struct containing a boolean (success) and log (output).
    - JSON Response: Serializes the result for HTTP communication with the React frontend.

Integration with the Workshop

The verifier_validate function is triggered by the React frontend’s “Execute Step 4” button, which sends the VP JWT (from Step 3) to the /verifier/validate endpoint. It relies on:

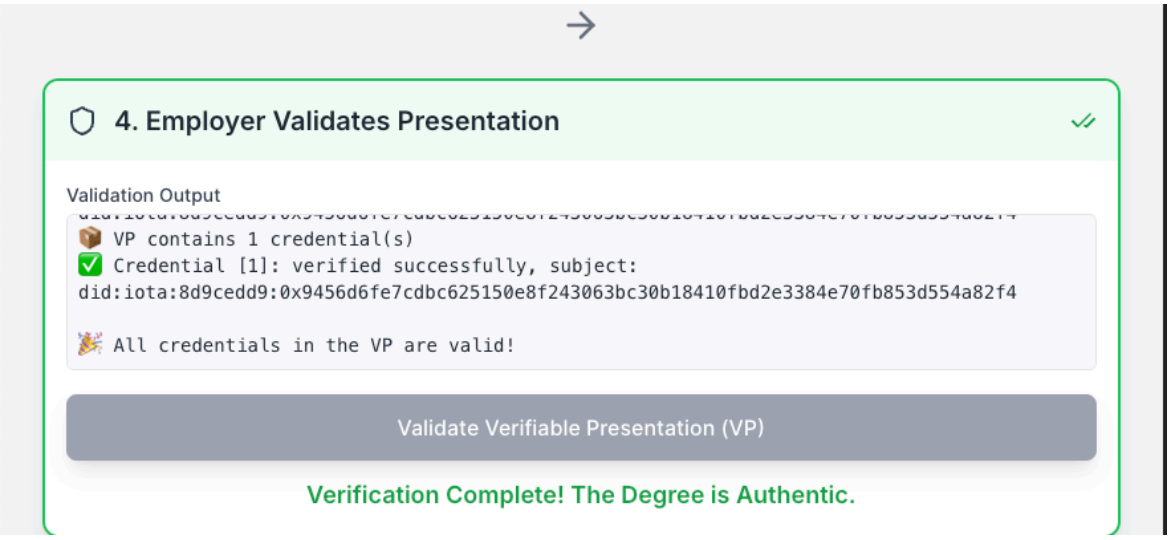
- Input: The VP JWT (body.vp_jwt) from the frontend.
- Dependencies: get_read_only_client for network access and the Resolver for fetching Decentralized Identifier documents.
- Prerequisites: The Student's Decentralized Identifier (Step 1) and Verifiable Credential (Step 2) must exist, and the VP (Step 3) must be valid.

The returned ValidationRe
validation outcome and lo

This website uses cookies

We use cookies to improve your experience. You can manage your preferences below. Read our Cookie Policy

Output after a successful validation of the student's Verifiable Presentation.



Explanation:

This completes the guided workshop, demonstrating the seamless and robust interaction between the React frontend and the Rust Axum backend across all four phases of the IOTA Identity flow.

Run and Test the Project

Now that you've set up the prerequisites (Rust, Node.js, VS Code) and copied the scaffold code Scaffold Code, it's time to run the Rust backend and React frontend, then test the IOTA Identity workflow. This section guides you through launching both applications and verifying the University Degree Verification system, which creates a Decentralized Identifier (DID), issues a Verifiable Credential (VC), creates a Verifiable Presentation (VP), and verifies it.

macOS/Linux Windows

Run the Backend (Rust)

1. **Navigate to the Backend Directory**:
   - Open a terminal a

     ```
     cd iota-identi
     ```

2. **Install Dependencies**
   - Build the Rust pro

     ```
     cargo build
     ```

This website uses cookies

We use cookies to improve your experience. You can manage your preferences below. Read our Cookie Policy

3. **Set the Environment Variable**:
   - Set the `IOTA_IDENTITY_PKG_ID` to a unique identifier (e.g., `workshop-123`) for your workshop session:

     ```
     export IOTA_IDENTITY_PKG_ID="workshop-123"
     ```

4. **Start the Backend Server**:
   - Run the Axum server:

     ```
     cargo run
     ```

   - The server starts at `http://localhost:3001`. You should see:

     ```
     IOTA Identity Axum API Server running on http://0.0.0.0:3001
     ```

   - Keep this terminal open.

Run the Frontend (React)

1. **Open a New Terminal**:
   - Open a second terminal window (e.g., press `Cmd + T` in your terminal).
2. **Navigate to the Frontend Directory**:
   - Go to the frontend folder:

     ```
     cd iota-identity-frontend
     ```

3. **Install Dependencies**:
   - Install Node.js dependencies:

     ```
     npm install
     ```

4. **Start the Frontend**:
   - Launch the Reac

     ```
     npm run dev
     ```

   - The app starts at
     use). The termina
   - Open `http://loc`

This website uses cookies

We use cookies to improve your experience. You can manage your preferences below. Read our Cookie Policy

Test the Project

1. **Access the Workshop UI**:
   - In your browser, go to `http://localhost:5173`.
   - Verify the workshop interface loads, showing the "University Degree Verification dApp " title and four steps (Student Creates Identity (Decentralized Identifier) , University Issues Degree to Student (Verifiable Credential) , Student Shows Degree To Employer (Verifiable Presentation) , Employer Confirms Validity of Degree).
2. **Configure Settings**:
   - In the "Connection Settings" section:
     - Set **IOTA Package ID** to `workshop-123` (same as the backend).
     - Set **API Base URL** to `http://localhost:3001/api`.
   - Click outside the input fields to save.
3. **Run the Workflow**:
   - **Step 1: Student Creates Identity (Decentralized Identifier)**:
     - Click "Create Student Identity".
     - Verify a DID appears (e.g., `did:iota:abc123...`) in the "Student DID (did:iota:...)" field.
   - **Step 2: University Issues Degree to Student (Verifiable Credential)**:
     - Click "Issue Degree".
     - Check for a JWT in the "Verifiable Credential (VC JWT)" field.
   - **Step 3: Student Shows Degree To Employer (Verifiable Presentation)**:
     - Click "Show Degree To Employer".
     - Confirm a JWT appears in the "Verifiable Presentation (VP JWT)" field.
   - **Step 4: Employer Confirms Validity of Degree**:
     - Click "Validate Degree".
     - Ensure the "Validation Output" field shows a success message (e.g., "Verification Complete! The Degree is Authentic.").
4. **Check for Errors**:
   - If an error appea
     terminal for logs
   - Ensure the backe
     both frontend an
   - Open your brows
     errors.

This website uses cookies

We use cookies to improve your experience. You can manage your preferences below. Read our Cookie Policy

5. **Verify End-to-End Flow**:

  - Confirm all four steps complete without errors, and the final step shows a green checkmark with the success message.
  - This indicates the degree was issued, presented, and verified successfully using IOTA Identity.

Once these steps are complete, you've successfully run and tested the workshop! You can reset the flow using the "Reset Flow" button to try again.

What You've Accomplished

You've built a full-stack IOTA Identity application:

- Created a Decentralized Identifier for a student (Holder).
- Issued a tamper-proof degree credential (VC).
- Packaged the Verifiable Credential into a signed Verifiable Presentation.
- Verified the Verifiable Presentation and Verifiable Credential on the IOTA network.

Extension tasks

Build a solution based on any of the IOTA Identity use cases — such as decentralized digital identity ownership, verifiable product credentials, or digital asset provenance — or explore an entirely new idea that leverages IOTA Identity's verifiable credentials and decentralized identifiers (DIDs). Once you've built your prototype, share it in the #identity channel on Discord to get feedback directly from the IOTA team and community.

Give Feedback

Tags: Tutorial Rust

« IOTA Notarization W

This website uses cookies

We use cookies to improve your experience. You can manage your preferences below. Read our Cookie Policy